

Centralized Resource Brokerage for Multi-Cluster Kubernetes: A Secure, Reservation-Based Architecture with Automatic Ligo Peering

Mohammadmahdi Azizian
Politecnico di Torino
Turin, Italy
mmahdiazizian@gmail.com

Abstract—Organizations often run many Kubernetes clusters at the same time. This creates a common problem: some clusters are busy, while others have free CPU and memory that nobody uses. Today there is no simple and safe way to share resources between clusters, so operators add extra capacity to each one and average usage stays low. This thesis presents a system that closes this gap. A central broker collects resource information from every cluster through a lightweight agent. When a user asks for resources, the broker selects the best provider and locks its capacity, so the same resources are never given to two requests. All traffic uses mutual TLS, and each cluster is identified by its certificate. After the decision, the agent connects the two clusters automatically with Ligo. Experiments show that the broker serves 100 clusters on a single CPU core, answers each request in under one second, and never double-books capacity.

Index Terms—Kubernetes, multi-cluster, resource sharing, Ligo, mutual TLS, edge computing, reservation, broker

I. INTRODUCTION

Containers have become the standard way to deploy software, and Kubernetes is the most common platform to run them. As companies grow, they usually do not run a single cluster. Instead, they run several clusters across data centers, cloud providers, and edge sites. They do this for many reasons: to stay close to users, to follow data rules, to isolate failures, and to give each team its own environment.

This model brings a clear downside, which is *resource fragmentation*. At any moment, some clusters are under heavy load while others sit almost idle. Because each cluster is managed on its own, operators add extra capacity to every cluster to handle the worst case. Studies report that average use of CPU and memory often stays between 20% and 50% [1], [2]. This wastes hardware, money, and energy.

The idea of a *computing continuum*, where resources flow freely from the edge to the cloud, has gained attention in research, for example in the European FLUIDOS project [3]. To reach this idea, we need a way to find, match, and assign resources across clusters in a safe and consistent way.

Master's thesis summary. Supervisors: Prof. Fulvio Risso, Dr. Stefano Galantino, and Dr. Jacopo Marino. This work was developed in collaboration with Engineering Ingegneria Informatica S.p.A. within the European project IPCEI-CIS AVANT.

This thesis presents such a system. It uses a central broker, lightweight agents in each cluster, security based on mutual TLS, and automatic cluster connection through Ligo.

The key design choice is a *synchronous* reservation. The cluster that asks for resources receives the answer inside the same HTTP request, in under one second. The chosen provider learns about its task a few seconds later, after a short poll. This single choice is what makes the system feel fast, and it is a main theme of the work.

II. PROBLEM AND BACKGROUND

A Kubernetes cluster has a control plane and several worker nodes. The scheduler places pods on nodes, but it only sees the nodes of its own cluster. There is no shared view across clusters. From this, four problems follow.

No global view. No component knows which clusters have free capacity and which are under pressure.

No reservation. Even if free capacity is found, there is no way to reserve it for a remote user and stop others from taking the same capacity. This is a classic double-booking problem.

Security at the border. Clusters often belong to different teams or companies. Any communication between them must be authenticated, without sharing passwords or a common login system.

From decision to action. Choosing a provider is only half of the work. The system must also create real network connectivity so that workloads can move.

Existing tools solve only parts of this. Kubernetes Federation (KubeFed) [4] spreads workloads over clusters, but it has no resource market and no reservation. Ligo [5] connects two clusters and lets one offload pods to the other, but the peering is set up by hand and there is no central decision and no reservation. So none of these tools prevents double-booking when several users target the same provider. This thesis fills that gap.

III. SYSTEM DESIGN

The system uses a *hub-and-spoke* design, shown in Fig. 1. A central broker is the hub, and one agent runs in each cluster. All agent-broker traffic goes over HTTPS with mutual TLS.

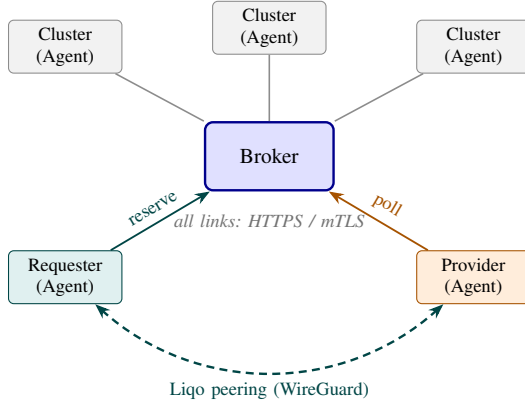


Fig. 1. Hub-and-spoke architecture. Many clusters connect to one broker over mutual TLS, each through a lightweight agent. For a single request the broker selects a provider, and the requester peers with it through Ligo. Any cluster can act as requester or provider.

Data model. The system stores its state as Kubernetes Custom Resources (CRDs). The main ones are the *ClusterAdvertisement* (what a cluster offers), the *Reservation* (a request handled by the broker), and two small *instruction* objects that tell each side what to do. Using CRDs lets the system reuse Kubernetes features such as storage, access control, and change notifications.

Resource calculation. Each agent computes the free capacity of its cluster with a simple formula:

$$\text{Available} = \text{Allocatable} - \text{Allocated} - \text{Reserved}. \quad (1)$$

Allocatable is the capacity that can host pods, *Allocated* is the sum of the requests of running pods, and *Reserved* is the capacity locked by the broker for other clusters.

The Reserved field. The *Reserved* value in (1) is the key to consistency. When the broker selects a provider, it adds the requested amount to that provider's *Reserved* field before it confirms the reservation. The next request then sees less free capacity. Because this update uses optimistic concurrency control (a safe retry on conflict), two requests can never lock the same capacity. The agent is careful to keep this field when it sends a new advertisement, so it does not erase the broker's locks.

Decision engine. For each request, the broker runs a simple algorithm with three steps. First it *filters* out clusters that are the requester itself, are not active, or do not have enough free CPU or memory. Then it *scores* the rest: a cluster gets a higher score when it keeps more free capacity after the request. Finally it *selects* the cluster with the highest score. This spreads the load and avoids filling one cluster too fast.

Security. The system uses mutual TLS for every call. A small certificate authority, managed by cert-manager, signs one certificate per cluster. The identity of a cluster is simply the Common Name in its certificate. This removes the need for tokens, API keys, or a shared login service.

Synchronous reservation. The requester sends one HTTP call to the broker. The broker runs the decision, locks the

capacity, and returns the answer in the same response. So the requester knows its provider in under one second, without polling. The provider does not need such a fast path, so it learns about its task with a light poll every five seconds. This split, shown in Fig. 2, keeps the design simple and the requester fast.

Ligo peering. After the requester receives its instruction, the agent runs `liqoctl peer` on its own. Ligo then creates a virtual node and an encrypted WireGuard tunnel. From this point, normal Kubernetes scheduling can place pods on the virtual node, and Ligo moves them to the provider. No manual step is needed.

IV. IMPLEMENTATION

The system is written in Go and uses the standard Kubernetes tools: controller-runtime for the reconcile loops, Kube-builder for the CRD code, and cert-manager for certificates. The broker and the agent are two separate programs.

The **broker** exposes a small REST API built on Go's `net/http` with a TLS configuration that requires a valid client certificate. A middleware reads the cluster identity from the certificate and blocks any cluster that tries to publish data under another name. A reservation controller drives each reservation through its lifecycle (Pending, Reserved, Active, Released, or Failed) and frees the locked capacity when a reservation ends.

The **agent** has a metrics collector that reads nodes and pods and applies (1). An mTLS HTTP client sends advertisements and reservation requests; before each advertisement it first reads the current *Reserved* value from the broker and keeps it. The client also retries failed calls with exponential backoff. A light poller checks for provider instructions every five seconds, and a separate controller starts the Ligo peering. The communication layer is hidden behind an interface, so a future transport such as gRPC could be added without changing the controllers. The full system is released as open-source software.

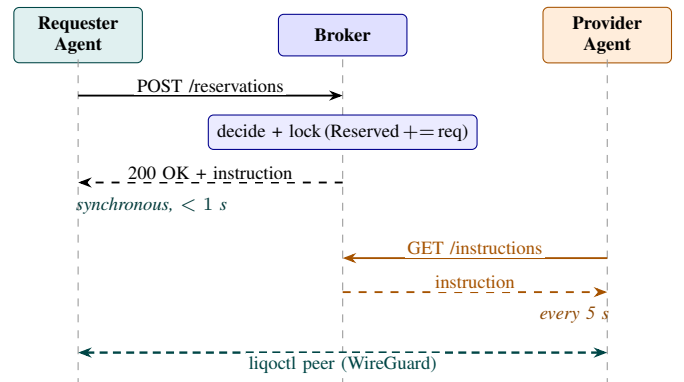


Fig. 2. Reservation flow. The requester receives its answer in the same HTTP response (synchronous, under one second). The provider discovers its task by polling every five seconds. The requester then peers with the provider through Ligo.

V. EVALUATION

The system was tested on a server with 96 CPU cores and 503 GB of RAM, using Kind (Kubernetes in Docker) to create the clusters. Seven automated tests measure scalability, bandwidth, agent cost, latency, behavior under concurrency, decision quality, and startup time. Each test is a script that starts every component, runs the experiment, and saves the results.

Scalability. Fig. 3 shows the broker CPU as the number of agents grows from 1 to 100. The cost grows in a roughly linear way up to about 40 agents, and then it settles on a plateau near 27% of one core. The plateau means the broker is not overloaded, even with 100 clusters. Memory stays almost constant near 40 MB, because the data lives in etcd and not in the process.

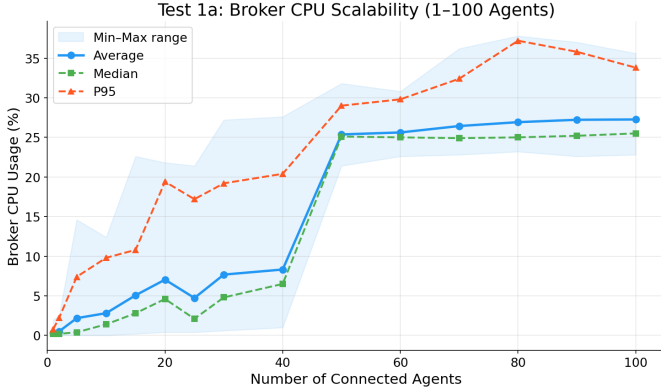


Fig. 3. Broker CPU use against the number of connected agents. The cost is linear up to about 40 agents and then stays near 27% of one core.

Latency. For ten reservations, the broker makes its decision in about 433 ms on average. The requester receives its instruction in about 526 ms, so the answer is well under one second. The provider learns about its task in about 2.2 s, which stays inside the five-second poll window. Because the requester no longer waits for a poll cycle, this is more than an order of magnitude faster than a fully poll-based design, where both sides waited for 30-second cycles.

Concurrency. The system was tested with 1 to 10 reservations sent at the same time, with five repeats per level. The median time grows from about 200 ms to about 795 ms and then flattens, because the Kubernetes API writes are serialized. Across all 30 runs there were *zero* timeouts and *zero* double-bookings. This confirms that the *Reserved* field keeps the system consistent under load.

Decision quality. Five scenarios with known capacities were run five times each. In every case the broker chose the cluster with the most free capacity and correctly rejected a request that no cluster could satisfy. So the engine picked the best provider in every tested case.

Agent cost and startup. A single agent uses about 40 MB of memory and about 0.3% of one CPU core. It sends only about 506 bytes per second (about 148 KB over five minutes), so it fits well on edge nodes with limited bandwidth. A warm

start takes one to four seconds, and most of this time is the generation of the TLS certificate, not the agent itself.

Table I collects the main results.

TABLE I
MAIN EVALUATION RESULTS

Metric	Result
Broker (100 agents)	~27% of one CPU core; memory constant ~40 MB
Reservation latency	~433 ms decision; requester under 1 s; provider ~2.2 s
Concurrency (10)	~795 ms median; 0 timeouts; 0 double-bookings
Decision quality	best provider chosen in every scenario
Agent footprint	~40 MB RAM; ~0.3% CPU; ~506 B/s

VI. CONCLUSION

This thesis showed that a central broker, together with automatic cluster peering, is a practical way to share resources across many Kubernetes clusters. A central point makes consistent reservations easy, because every request passes through it and capacity is locked atomically. The cost of this central point is small: one broker handled 100 clusters on a single core with flat memory. The synchronous design gives the requester an answer in under one second, while the provider stays close behind with a light poll. The *Reserved* field prevents double-booking, and the Ligo integration turns a decision into resources that workloads can really use.

The work also has clear limits. The broker is a single point of failure; a production setup would need a highly available version. The provider side still uses polling, which a push channel could replace. The scoring engine looks only at CPU and memory, so it ignores network distance, cost, and special hardware such as GPUs. Finally, all tests ran on one machine, so a real multi-site deployment would add network delay. These points are natural directions for future work, including a highly available broker, push notifications, richer scheduling policies, and dynamic, token-based peering.

REFERENCES

- [1] A. Verma *et al.*, “Large-scale cluster management at Google with Borg,” in *Proc. EuroSys*, 2015, pp. 1–17.
- [2] E. Cortez *et al.*, “Resource Central: understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proc. SOSP*, 2017, pp. 153–167.
- [3] FLUIDOS Consortium, “FLUIDOS: Flexible, scalable, secure and decentralized operating system,” EU Horizon Europe project, 2024. [Online]. Available: <https://www.fluidos.eu/>
- [4] Kubernetes SIG Multicluster, “Kubernetes Cluster Federation (KubeFed),” 2023. [Online]. Available: <https://github.com/kubernetes-sigs/kubefed>
- [5] M. Iorio, F. Risso, A. Palesandro, L. Camiciotti, and A. Manzalini, “Computing without borders: the way towards liquid computing,” *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 2820–2838, 2023.
- [6] The Kubernetes Authors, “Kubernetes: production-grade container orchestration,” 2024. [Online]. Available: <https://kubernetes.io/>
- [7] The cert-manager Authors, “cert-manager: X.509 certificate management for Kubernetes,” 2024. [Online]. Available: <https://cert-manager.io/>